

When to retrain the flow in NESSAI: a renewal-theory decision model

Investigation notes, branch `claude/nessai-retrain-decision-pmm9kr`

September 26, 2026

Abstract

We build a simple, principled rule for deciding, before each nested-sampling iteration, whether to retrain (or reset) the normalising flow in NESSAI, with the aim of minimising total run time. Instrumented runs show that (i) the proposal pool is paid for when it is populated, so the only sensible decision points are when the pool is empty, and (ii) between trainings the acceptance decays as $\exp(-s/n_{\text{live}})$ with remarkable precision. Under these facts the problem is a classic renewal (machine-replacement) problem. It has a closed-form optimal long-run cost rate g^* , which answers the question of how far ahead to look: one training cycle, with everything beyond it summarised by g^* . The rule is to retrain when the next pool drawn from the current flow would cost more per iteration than g^* . Uncertainties are propagated at the Gaussian level and calibrated on the run’s own history. Costs are deterministic counts multiplied by unit costs, which are either provided by the user (deterministic schedule) or measured. On three test problems with six seeds each, the rule reduces run time relative to NESSAI’s default schedule by 16% when training dominates the cost and by 17% when the likelihood dominates (emulated $t_L = 10\text{ ms}$). In between it is neutral, and it is never significantly worse. A fixed schedule is up to $1.6\times$ slower. The reset option gains a further 12% at $t_L = 10\text{ ms}$, coming mostly from the bimodal problem.

1 Setting and empirical facts

In NESSAI’s standard nested sampler a pool of P points is drawn from the flow proposal. All P likelihoods are evaluated when the pool is populated, and each iteration then draws points from the pool until one lies above the current likelihood contour. Let

- c be the cost of one pool point (flow sampling, rejection sampling, likelihood evaluation, amortised loop overhead);
- T be the cost of training the flow;
- $a(s)$ be the acceptance s iterations after the last training, i.e. the probability that a pool point lies inside the current contour.

Each iteration consumes on average $1/a$ pool points, so the cost per iteration is $r(s) = c/a(s)$.

Fact 1: the pool is paid up front. `FlowProposal.train` discards the current pool. Retraining with a non-empty pool therefore throws away points whose cost has already been paid, while using up the pool costs (almost) nothing extra. Retrain decisions should only be taken when the pool is empty. Fixed-frequency schedules violate this: `training_frequency=500` with `train_on_empty=False` used $1.1\text{--}1.6\times$ the likelihood evaluations of the default (train-on-empty) schedule, for a similar number of trainings.

Fact 2: exponential decay at a known rate. If the flow density q is approximately constant over the region enclosed by the contour, the acceptance is $a = \int_{L>L^*} q d\theta \approx \bar{q} X$. The prior volume shrinks as $X \propto e^{-i/n_{\text{live}}}$, so

$$a(s) = A e^{-ks}, \quad k \simeq 1/n_{\text{live}}, \quad (1)$$

where A is the “fresh” acceptance of the newly trained flow. Figure 1 shows episodes of 3000 iterations without retraining on three problems. The fitted slopes are $kn_{\text{live}} = 1.00 \pm 0.04$, the linear model fits to within the counting noise, and the result holds even at the start of an episode. The exponential model is therefore exact in practice and the rate is known a priori. We still fit it, with a prior centred on $1/n_{\text{live}}$, because proposal schemes that shrink the proposal volume between trainings would change it.

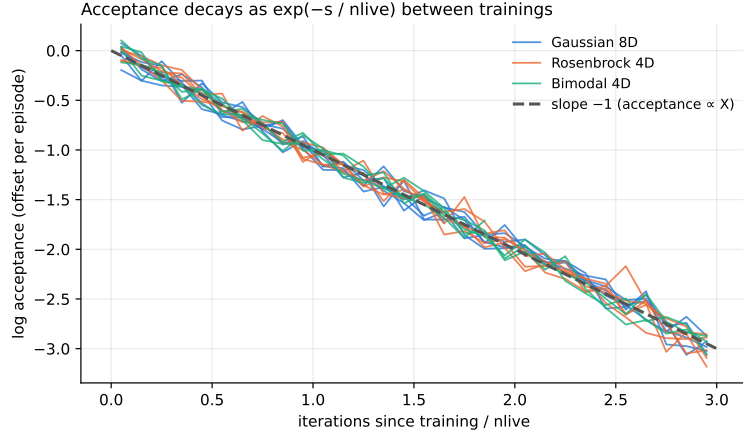


Figure 1: Binned log-acceptance in episodes of 3000 iterations between trainings ($n_{\text{live}} = 1000$). The dashed line has slope $-1/n_{\text{live}}$.

Fact 3: the fresh acceptance is a level, not a gain. The fresh acceptance A of a warm-started retrain does not depend on how long ago the previous training was, so a multiplicative “improvement” $A_{\text{new}}/a_{\text{before}}$ is not a stable quantity (it grows as $e^{k \text{ gap}}$). We therefore predict $\log A$ as a level estimated from recent episodes. It drifts slowly over the run, in either direction.

Fact 4: training cost. The time per epoch is very stable, and the number of epochs of a warm start grows mildly with the gap (about 30 at 500-iteration gaps vs. about 100 at 3000 on Rosenbrock/bimodal). A full reset costs 3–5× the epochs of a warm start.

2 The decision model

2.1 How far to look ahead

Consider two policies at a moment when the pool is empty:

- (R) retrain now, then act optimally;
- (C) draw one more pool of P points from the current flow, lasting n iterations, then retrain and act optimally.

The number of iterations N of a run is set by the stopping criterion, not by the schedule. After the extra block, (C) therefore follows the same process as (R), shifted by n iterations. If the long-run cost per iteration of the optimal schedule is g^* , then

$$\text{Cost}(C) - \text{Cost}(R) \longrightarrow cP - g^* n \quad (2)$$

as the remaining horizon grows. This is the constant to which the two exponentially growing cost curves converge. They only cancel if future retraining is allowed in both branches; if it is not, both costs diverge and their difference is dominated by the better flow, which always favours retraining. Equation (2) gives the rule

$$\boxed{\text{retrain} \iff cP > g^* \mathbb{E}[n]} \quad (3)$$

i.e. retrain when continuing costs more per iteration than the optimal long-run rate. The effective horizon is one training cycle.

2.2 The optimal long-run rate

A cycle that starts with a training and lasts τ iterations costs

$$C(\tau) = T + \int_0^\tau \frac{c}{A'} e^{ks} ds = T + \frac{c}{A'k} (e^{k\tau} - 1), \quad (4)$$

where A' is the fresh acceptance of the next flow. By the renewal-reward theorem the long-run cost per iteration is $g(\tau) = C(\tau)/\tau$.

Proposition 1. *Let $y = TkA'/c$ (the training cost in units of the cost of $1/k$ iterations of a fresh flow). The minimum $g^* = \min_\tau g(\tau)$ is $g^* = xc/A'$, reached at $\tau^* = \ln(x)/k$, where $x \geq 1$ is the unique solution of*

$$x \ln x - x + 1 = y. \quad (5)$$

Equivalently, g^* satisfies

$$T = \int_0^\infty \left(g^* - \frac{c}{A'} e^{ks} \right)^+ ds. \quad (6)$$

Proof. $g'(\tau) = 0 \iff \tau C'(\tau) = C(\tau)$. Since $C'(\tau) = ce^{k\tau}/A'$, at the optimum the instantaneous rate equals the average rate: $ce^{k\tau^*}/A' = g^*$. Writing $x = e^{k\tau^*}$, the condition $C(\tau^*) = g^*\tau^*$ becomes $T + \frac{c}{A'k}(x-1) = \frac{c}{A'}x \frac{\ln x}{k}$, which is (5). The left-hand side of (5) is increasing for $x > 1$ and vanishes at $x = 1$, so the solution is unique. For (6), $\int_0^{\tau^*} (g^* - ce^{ks}/A') ds = g^*\tau^* - (C(\tau^*) - T) = T$. $\square$

Equation (6) has a simple reading: the training cost is recovered by the savings accumulated while the new flow runs below the break-even rate (Fig. 2). Limits:

$$y \ll 1: \quad x \simeq 1 + \sqrt{2y}, \quad \tau^* \simeq \sqrt{\frac{2TA'n_{\text{live}}}{c}}; \quad y \gg 1: \quad x \simeq \frac{y}{\ln y}. \quad (7)$$

The small- y limit is a square-root law, analogous to the economic order quantity. Equation (5) is solved by bisection.

Training cost growing with the gap. If $T = T_0 + T_1\tau$, then $g(\tau) = T_1 + [T_0 + \frac{c}{A'k}(e^{k\tau} - 1)]/\tau$, and in (3) continuing for n iterations also adds T_1n to the next training. T_1 cancels, so only the fixed part T_0 matters. Using the mean training cost (as we do) slightly overestimates T_0 and is mildly conservative.

2.3 Pool yield, pool planning and finite horizon

Yield. A pool of P points drawn s iterations after training lasts n iterations, where $\int_s^{s+n} e^{ks'} ds'/A = P$, i.e.

$$n = \frac{1}{k} \ln(1 + Pk a(s)). \quad (8)$$

With NESSAI's default pool size $P \approx n_{\text{live}}/\bar{a}$ a pool lasts roughly $0.3\text{--}0.7 n_{\text{live}}$ iterations.

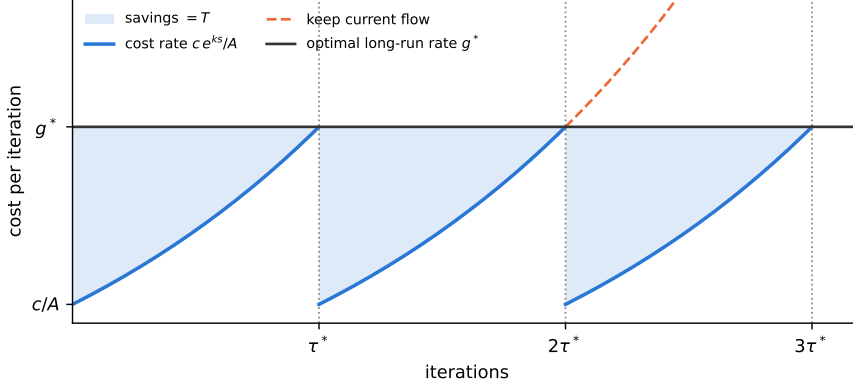


Figure 2: Cost per iteration under the optimal policy. Each shaded area equals T ; keeping the current flow past τ^* (dashed) costs more than g^* per iteration.

Pool planning. Since decisions happen when the pool is empty, the schedule cannot retrain more often than once per pool. When τ^* is shorter than a pool (expensive likelihoods), we cap the next pool at the number of points needed to reach the break-even point $c/a(s') = g^*$:

$$P_{\text{plan}} = \frac{1}{k} \left(\frac{g^*}{c} - \frac{1}{a(s)} \right), \quad (9)$$

with a floor corresponding to $\max(n_{\text{live}}/10, \text{bin size})$ iterations. After a retrain the same formula is used with $a = A'$. There is no penalty for smaller pools beyond a small fixed overhead per population.

Finite horizon. We estimate the remaining iterations from the stopping criterion. With $\Delta \log Z = \log(1 + L_{\text{max}}X/Z)$ and $X \propto e^{-i/n_{\text{live}}}$, $N_{\text{rem}} \simeq n_{\text{live}}[\log(e^{\Delta \log Z} - 1) - \log(e^\epsilon - 1)]$. If $N_{\text{rem}} < \tau^*$ we compare the expected costs to the end directly, without further retraining: $T + \frac{c}{A'k}(e^{kN_{\text{rem}}} - 1)$ against $\frac{c}{a(s)k}(e^{kN_{\text{rem}}} - 1)$.

2.4 Resetting the flow

A reset replaces warm-start training by training from scratch, with cost T_r and fresh acceptance A_r . We model $\log A_r = \log A_w + \delta$ and compute g_r^* from (5) with (T_r, A_r) . When retraining, we reset if $g_r^* < g_w^*$. The posterior on δ is Gaussian, combining a prior $\mathcal{N}(0, 1)$ with two kinds of evidence:

- *lineage degradation*: $\log A$ of the first (from-scratch) episode of the current lineage minus the current warm prediction, with variance $\sigma_0^2 + (\gamma \Delta i / n_{\text{live}})^2$. Its relevance decays as the posterior changes ($\sigma_0 = 0.3$, $\gamma = 0.05$);
- *paired outcomes of past resets*: $\log A$ of each reset episode minus that of the preceding episode, with variance $2\sigma_0^2$.

A lineage that has degraded since it started favours a reset; a lineage that has improved, or a reset that did not help, counts against one. With no evidence, $\mathbb{E}[\delta] = 0$ and the larger T_r prevents resets.

3 Estimating the inputs with uncertainties

Current flow. The draws needed per iteration are geometric, $m_i \sim \text{Geom}(a_i)$. Binning b iterations gives $\hat{a} = b / \sum m_i$ with, by the delta method,

$$\text{Var}[\log \hat{a}] \simeq \frac{1 - a}{b}. \quad (10)$$

We fit $\log a(s) = \alpha + \beta s$ by weighted least squares with a conjugate Gaussian prior, which gives a closed-form Gaussian posterior on (α, β) . The prior on α is the fresh-level prediction below, and the prior on β is the precision-weighted mean of recent episodes (initially $-1/n_{\text{live}} \pm 30\%$). The bin size is $\max(10, n_{\text{live}}/20)$.

Fresh flow. For every finished episode we store a data-only fit $\widehat{\log A_j}$. The prediction for the next warm retrain is the mean of the last $m = 5$ episodes of the current lineage. Its predictive variance is calibrated on the run’s own past prediction errors e_j :

$$\sigma_{A'}^2 = \frac{w_0 \sigma_p^2 + \sum_j e_j^2}{w_0 + N_e}, \quad \sigma_p = 0.3, \quad w_0 = 2. \quad (11)$$

This replaced a sample variance over the last five episodes, whose error bars were too narrow (Table 1).

Propagation. Costs are convex in $1/a$, so uncertainties raise expected costs through log-normal moments:

$$\mathbb{E}[1/A'] = e^{-\mu + \sigma^2/2}, \quad \mathbb{E}[1/k] \simeq \frac{1}{\hat{k}} \left(1 + \frac{\sigma_k^2}{\hat{k}^2} \right), \quad (12)$$

which enter (5) through $A_{\text{eff}} = 1/\mathbb{E}[1/A']$ and $k_{\text{eff}} = 1/\mathbb{E}[1/k]$. $\mathbb{E}[n]$ is computed by 7-point Gauss–Hermite quadrature over the posterior of $\log a(s)$. c and T enter linearly, so only their means affect the expected cost.

Table 1: Calibration of the Gaussian predictions (z-scores; the ideal is mean 0, sd 1 and 4.6% beyond 2σ).

prediction	N	mean z	sd z	$ z > 2$
log a at the start of the next pool	648	−0.06	1.03	5.2%
pool yield n (log ratio realised/predicted)	648	bias −2%, scatter 7%		
fresh log A' , sample variance of 5 episodes	727	−0.4 ... + 0.3	1.2–1.8	15–24%
fresh log A' , calibrated variance	727	−0.2 ... + 0.2	0.87–1.12	3.4–6.5%

4 Costs and determinism

All costs are deterministic counts multiplied by three unit costs: likelihood evaluations $\times$ [s/evaluation], pool points $\times$ [s/point], and epochs $\times$ training samples $\times$ [s/(sample-epoch)]. For a fixed seed the epoch counts are deterministic, since early stopping depends only on the data and the seed. With `retrain_costs=dict(likelihood=..., population=..., training=...)`, or the path of a previous run’s `retrain_costs.json`, the schedule is fully deterministic. In the benchmark, decision runs whose costs make them retrain at every empty pool reproduced the default schedule’s trajectory exactly. Missing unit costs are measured during the run, which makes the schedule depend on the hardware. Measured costs are always tracked:

- likelihood time comes from the model’s counters;
- population cost is the non-training sampling time minus likelihood time, per pool point;
- training cost is the wall time per sample-epoch.

A warning is logged if a measured cost differs from the provided value by more than $2\times$ (after three measurements), and the measured costs are written to `retrain_costs.json` at the end of the run.

5 Experiments

Setup. Three problems: an 8D Gaussian, 4D Rosenbrock and a 4D bimodal correlated Gaussian. $n_{\text{live}} = 1000$, the default **FlowProposal**, and a flow with 4 blocks of 2×16 neurons; CPU, one thread per run. The test likelihoods are nearly free ($0.1 \mu\text{s}$), so we emulate expensive ones with a *virtual* cost t_L :

$$\text{virtual time} = \text{wall time} + N_{\text{like}} t_L, \quad t_L \in \{0, 10^{-4}, 10^{-3}, 10^{-2}\} \text{ s.}$$

Decision runs receive t_L through **retrain_costs**, together with the measured unit costs for population ($75 \mu\text{s}/\text{point}$) and training ($14 \mu\text{s}/\text{sample-epoch}$). The dimensionless training cost y spans about 2 ($t_L = 0$) down to about 0.015 ($t_L = 10^{-2}$), and τ^* spans about $1.3n_{\text{live}}$ down to $0.16n_{\text{live}}$. Policies:

- *default*: train on empty, cooldown 200, retrain when acceptance < 0.01 ;
- *fixed*: every 1000 iterations;
- *decision*: rule (3) with pool planning, with provided costs, or with measured costs (*dyn*);
- *decision + reset*.

Each (problem, policy, t_L) is run with 6 seeds. Comparisons are paired by problem and seed and summarised by the geometric mean of the time ratio to *default*, with a 95% bootstrap interval.

5.1 Results

Table 2: Run time relative to the default schedule (geometric mean over 3 problems $\times$ 6 seeds, 95% bootstrap interval), mean number of trainings and resets.

t_L [s]	policy	time ratio	trainings	resets
0	default	1	27.3	0
	fixed (1000 it)	0.88 [0.83, 0.93]	14.8	0.0
	decision	0.84 [0.78, 0.90]	11.0	0.0
	decision (measured costs)	0.82 [0.77, 0.88]	11.2	0.0
	decision + reset	0.85 [0.79, 0.91]	10.5	0.8
0.0001	default	1	27.3	0
	fixed (1000 it)	1.05 [1.01, 1.08]	14.8	0.0
	decision	0.94 [0.90, 0.98]	14.9	0.0
	decision + reset	0.95 [0.88, 1.01]	13.9	0.7
0.001	default	1	27.3	0
	fixed (1000 it)	1.41 [1.34, 1.47]	14.8	0.0
	decision	0.99 [0.94, 1.04]	26.1	0.0
	decision + reset	0.92 [0.81, 1.03]	24.1	0.8
0.01	default	1	27.3	0
	fixed (1000 it)	1.57 [1.48, 1.65]	14.8	0.0
	decision	0.83 [0.78, 0.87]	59.5	0.0
	decision + reset	0.73 [0.63, 0.84]	55.3	1.2

Tables 2 and 3 and Fig. 3 show the following.

- **Training-dominated regime** ($t_L \approx 0$, $y \approx 2$): the decision takes 0.84 [0.78, 0.90] of the default’s time. It trains 11 times instead of 27, with $\tau^* \approx 1.3n_{\text{live}}$. Measuring the costs during the run instead of providing them gives the same result (0.82).

Table 3: Time ratio to default per problem (geometric mean over 6 seeds).

t_L [s]	policy	Gaussian 8D	Rosenbrock 4D	bimodal 4D
0	decision	0.90	0.86	0.77
0	decision + reset	0.90	0.87	0.79
0	fixed (1000 it)	0.86	0.94	0.84
0.0001	decision	1.00	0.91	0.91
0.0001	decision + reset	1.00	0.97	0.89
0.0001	fixed (1000 it)	1.07	1.06	1.01
0.001	decision	0.92	1.07	0.98
0.001	decision + reset	0.97	0.98	0.83
0.001	fixed (1000 it)	1.48	1.39	1.35
0.01	decision	0.74	0.90	0.85
0.01	decision + reset	0.79	0.81	0.62
0.01	fixed (1000 it)	1.64	1.58	1.49

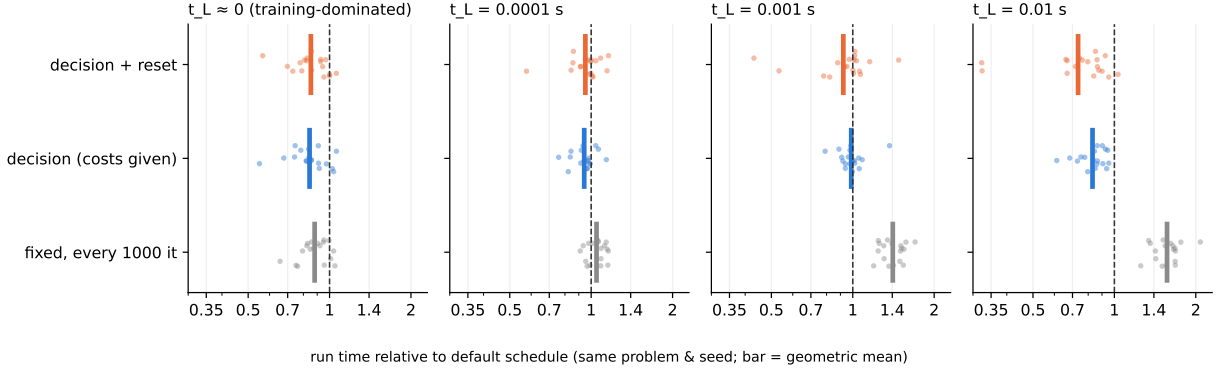


Figure 3: Run time of each run relative to the default schedule on the same problem and seed (dots), with the geometric mean (bars). Panels are for increasing emulated likelihood cost.

- **Intermediate regime** ($t_L = 10^{-4}, 10^{-3}$ s): 0.94 [0.90, 0.99] and 0.99 [0.94, 1.04]. Here τ^* is close to the default pool length ($0.3\text{--}0.7n_{\text{live}}$), so the default train-on-empty schedule happens to be near-optimal and there is little to gain.
- **Likelihood-dominated regime** ($t_L = 10^{-2}$ s, $y \approx 0.015$): 0.83 [0.78, 0.87]. $\tau^* \approx 0.16n_{\text{live}}$ is shorter than a pool, and pool planning lets the rule retrain about 60 times instead of 27. Without pool planning (earlier batch) the rule matched the default exactly: it retrained at every empty pool and reproduced the default trajectory.
- **Fixed schedules** discard partially used pools. Every 1000 iterations is competitive only when training dominates, and costs $1.41\text{--}1.57\times$ the default when likelihoods are expensive.
- **Reset.** The reset option triggers 0.7–1.2 times per run. It is neutral on the unimodal problems and helps on the bimodal one, where warm-started lineages sometimes degrade (0.62 vs. 0.85 for the plain decision at $t_L = 10^{-2}$; 0.83 vs. 0.98 at 10^{-3}). Overall, 0.73 [0.63, 0.84] at $t_L = 10^{-2}$ and consistent with the plain decision elsewhere. Resets pay off more when the likelihood is expensive, because the gain in acceptance is then worth more than the 3–5 $\times$ higher training cost.
- **Evidence.** $\log Z$ is unaffected. For the Gaussian, all policies give -16.61 ± 0.10 (analytic -16.614); for the bimodal, -10.59 ± 0.09 (analytic -10.604), with the same scatter as the default.

6 Discussion and limitations

- **Test problems.** The benchmark problems are small, and training dominates wall time unless t_L is large. The structure of the rule does not depend on this, since everything enters through $y = TkA'/c$. Confirming the gains on production problems (e.g. GW likelihoods with the group-mixture flows of this fork) is the obvious next step.
- **Trajectory noise.** Seed-to-seed variance is large. Warm-start lineages can drift to a $3\times$ different likelihood count on the same problem, which dominates single-run comparisons.
- **Pool granularity.** Without pool planning the rule can only choose among multiples of the pool length, and for expensive likelihoods it cannot improve on the default.
- **Resetting** is an exploration problem: we only learn what a reset gives by resetting. The lineage-relative evidence avoids the worst failure (repeatedly resetting into worse flows), but its prior parameters (σ_0, γ) are set by hand from a handful of runs.
- **Model assumptions.** Training outcomes are treated as independent of the gap, apart from the mean cost, and the future is assumed stationary within one cycle; slow drifts are tracked by re-estimating at every decision. The population unit cost absorbs the per-iteration loop overhead.

Usage.

```
FlowSampler(model, ...,
             retrain_decision=True,          # or dict(allow_reset=True, ...)
             retrain_costs="previous_run/retrain_costs.json")
```

Code: `src/nessai/samplers/retrain.py`. Experiments and scripts: `examples/retrain_decision/`.